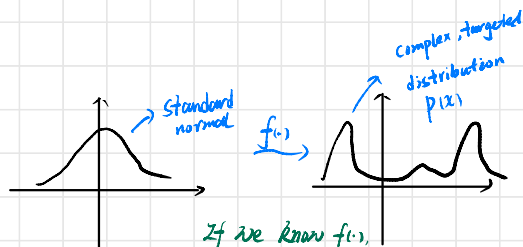
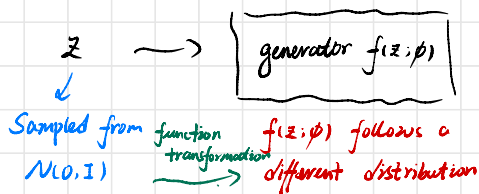


Normalizing flow tries to learn the distribution of the image *directly*.



Denote it as $pr(x|\phi)$

If we know this, then we just need to maximize the likelihood of real images.

If we know $f(\cdot)$,
Can we know $p(x)$?

→ generally not
If f is invertible, then yes

$$\text{Training: } \hat{\phi} = \arg \max_{\phi} \sum_{i \in \text{real images}} \log pr(x_i | \phi)$$

Hard part: what is $pr(x|\phi)$?

Integration by Substitution

$X \sim F(x)$, $Y = \phi(x)$, where ϕ is invertible function. $Y \sim ?$

$$Pr(Y \in R) = Pr(X \in \phi^{-1}(R))$$

$$= \int_{\phi^{-1}(R)} f_X(x) dx$$

$$= \int_R f_X(\phi^{-1}(y)) \left| \frac{d\phi^{-1}(y)}{dy} \right| dy \quad \text{Changing from } x \text{ to } y$$

$$\text{So } pr(x|\phi) = N(f^{-1}(x|\phi); 0, I) \left| \frac{df^{-1}(x)}{dx} \right| \rightarrow \text{absolute of the determinant}$$

$$= N(z; 0, I) \left| \frac{df(z)}{dz} \right|^{-1}$$

where $f(z; \phi) = x$

$$\det\left(\frac{\partial \phi^{-1}(y)}{\partial y}\right) = \det\left(\frac{\partial \phi(x)}{\partial x} \Big|_{x=\phi^{-1}(y)}\right)^{-1}$$

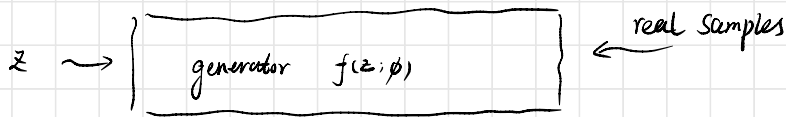
In order to be able to evaluate the likelihood, the generator $f(z; \phi)$

must be invertible. Note also we should be able to invert it and compute

determinant of the gradients efficiently.

↓
O(n³) operation for a invertible matrix.

← inverse mapping for evaluating likelihood of real samples → this direction for training



← forward mapping for generation → this direction for generation

Design of invertible layers.

→ this is the key in normalizing flow

- Desired property
1. the layer should be sufficiently expressive to approximate any function.
 2. must be invertible
 3. practically, should be able to invert and evaluate the determinant of the Jacobian efficiently.

Basis: Linear and elementwise flows

→ they satisfy property 2 and 3 but fail 1.
However they are the basis for more sophisticated layer satisfying all 1, 2 and 3 properties.

1. Linear flows

$$Y = \Omega x + \beta$$

→ if this Ω is invertible then this layer is invertible

design: $\Omega = \underbrace{P}_{\text{permutation matrix}} \underbrace{L}_{\text{lower triangular matrix with 1 on the diagonal}} \underbrace{U}_{\text{upper triangular matrix with non-zero diagonal}}$

To make sure Ω is invertible during training

1. P is fixed permutation matrix

2. the diagonal of L is fixed to be 1 during training.

3. to make sure there is no-zero in the diagonal of U

We do

$$\text{sign}(s_i) \cdot \exp(\alpha_i)$$

this is fixed at the beginning

this is initialized as $\log(1,1)$

$$\rightarrow \text{So } |\Omega| = |P||L||U|$$

$$\downarrow \quad \downarrow \quad \downarrow$$

$$1 \quad 1 \quad \prod_i \text{sign}(s_i) \exp(\alpha_i)$$

Initialization of Ω :

1. Generate a Random Orthogonal Matrix

Sample random matrix and then do QR decomposition

So it's determinant is 1
this will stabilize training at the beginning

2. Run PLU decomposition

diagonal all 1

3. $\Omega = PLU$

here notice that we replace the diagonal

$$\text{as } s_i = \text{sign}(s_i) \exp(\log(s_i, 1))$$

α_i : we train α_i here

Determinant of the Jacobian matrix

$$\frac{\partial y}{\partial x} = \Omega \Rightarrow \left| \frac{\partial y}{\partial x} \right| = |\Omega| = \det(P) \det(L) \det(U)$$

$$= (1) \cdot 1 \cdot \prod_i \text{sign}(s_i) \exp(\alpha_i)$$

Prod of diagonals

Inverse

$$Y = \Omega X + \beta$$

$$(Y - \beta) = \Omega X$$

$$\Rightarrow LUX = P^T(Y - \beta)$$

back substitution

$$LZ = P^T(Y - \beta)$$

$$UX = Z$$

both $O(n^2)$

Problem of linear flow: it is only linear, if we only use linear flow, we can only get linear combination of the input.

So we need non-linear part $\rightarrow$ element-wise flow

2. element-wise flow

$$y_i = f(x_i; \phi)$$

↪ invertible
non-linear
function

↪ e.g. leaky Relu
or exponential function

$$\begin{aligned} x_1 &\xrightarrow{f(\cdot; \phi)} y_1 \\ x_2 &\xrightarrow{\quad} y_2 \\ &\vdots \\ x_n &\xrightarrow{\quad} y_n \end{aligned}$$

Determinant of the Jacobian is simple:

$$|\det(J)| = \prod_{i=1}^D \left| \frac{df(x_i)}{dz_i} \right|$$

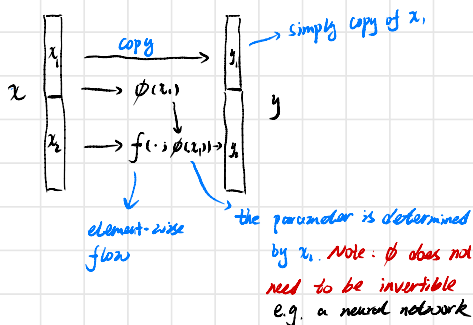
Inverse is simple if $f^{-1}(\cdot; \phi)$ is simple

problem of element-wise flow: it is non-linear but do not mix features

So it is natural to mix linear flow and element-wise flow

input $\rightarrow$ [linear flow $\rightarrow$ element-wise flow] $\rightarrow \dots \rightarrow$ [block] $\rightarrow$ output
↪ one block

3. Coupling flow $\rightarrow$ this uses element-wise flow



inverse process

$$x_1 = y_1$$

$$x_2 = f^{-1}(y_2, \phi(x_1))$$

Determinant of Jacobian

$$\begin{aligned} \begin{vmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{vmatrix} &= \begin{vmatrix} I & 0 \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{vmatrix} \\ &= \left| \frac{\partial y_2}{\partial x_2} \right| \end{aligned}$$

Standard normalizing flow: Learn a distribution easy to sample from (real images)

minimize $KL(q || p)$
 ↓
 target distribution
 ↘
 model

but hard to evaluate with.

$$\argmin_{\phi} KL(q || p) = \argmin_{\phi} \int q(x) \log \frac{q(x)}{p(x)} dx$$

$$= \argmin_{\phi} \left\{ \int q(x) \log q(x) dx - \int q(x) \log p_{\phi}(x) dx \right\}$$

$$= \argmax_{\phi} \int q(x) \log p_{\phi}(x) dx$$

$$= \argmax_{\phi} \mathbb{E}_{x \sim q} (\log p_{\phi}(x)) dx$$

$$\approx \argmax_{\phi} \frac{1}{n} \sum_{i=1}^n \log p_{\phi}(x_i) \Rightarrow \text{equivalent to maximize log-likelihood}$$

Reverse KL divergence:

Suppose we have a distribution we can evaluate it easily, but hard to sample from, now we want to approximate this distribution using normalizing flow so we can sample from this distribution.

minimize $KL(p_{\phi} || q)$
 ↙
 model
 ↘
 target distribution

$$\argmin_{\phi} KL(p_{\phi} || q) = \argmin_{\phi} \int p_{\phi}(x) \log \frac{p_{\phi}(x)}{q(x)} dx$$

$$= \argmin_{\phi} \left\{ \underbrace{\int p_{\phi}(x) \log p_{\phi}(x) dx}_{\text{entropy of } p_{\phi}(x)} - \int p_{\phi}(x) \log q(x) dx \right\}$$

$$= \argmin_{\phi} \left\{ \mathbb{E}_{x \sim p_{\phi}} (\log p_{\phi}(x)) - \mathbb{E}_{x \sim p_{\phi}} (\log q(x)) \right\}$$

$$\approx \argmin_{\phi} \left\{ \frac{1}{n} \sum_{i=1}^n \log p_{\phi}(\hat{x}_i) - \frac{1}{n} \sum_{i=1}^n \log q(\hat{x}_i) \right\}$$

Q: If we can evaluate the likelihood of the generated, why not just maximize the likelihood directly?

A: It would be equivalent to maximize this term. The result is

that all samples will collapse to the mode of $q(x)$. The entropy term prevents this from happening.

→ we still need to evaluate this likelihood like standard normalizing flow but we know z_i . So we don't need to invert the outputs

Consider Box transformation why we cannot maximize the likelihood of transformed data. Instead, we must maximize the likelihood of the observations.